



AMERICAN INTERNATIONAL UNIVERSITY-BANGLADESH

Faculty of Engineering

Lab Report

Experiment # 10

Experiment Title: Familiarization with the Raspberry Pi.

Date of Perform:	28 December 2025	Date of Submission:	5 January 2026
Course Title:	MICROPROCESSOR AND EMBEDDED SYSTEMS LAB		
Course Code:	EEE 4103	Section: [O]	
Semester:	Fall 2025-26	Degree Program:	BSc in CSE
Course Teacher:	Dr. Md. Rukonuzzaman		

Declaration and Statement of Authorship:

1. I/we hold a copy of this Assignment/Case Study, which can be produced if the original is lost/damaged.
2. This Assignment/Case Study is my/our original work and no part of it has been copied from any other student's work or from any other source except where due acknowledgment is made.
3. No part of this Assignment/Case Study has been written for me/us by any other person except where such collaboration has been authorized by the concerned teacher and is acknowledged in the assignment.
4. I/we have not previously submitted or currently submitting this work for any other course/unit.
5. This work may be reproduced, communicated, compared, and archived to detect plagiarism.
6. I/we permit a copy of my/our marked work to be retained by the Faculty Member for review by any internal/external examiners.
7. I/we understand that Plagiarism is the presentation of the work, idea, or creation of another person as though it is your own. It is a form of cheating and is a very serious academic offense that may lead to expulsion from the University. Plagiarized material can be drawn from, and presented in, written, graphic, and visual forms, including electronic data, and oral presentations. Plagiarism occurs when the origin of the source is not appropriately cited.
8. I/we also understand that enabling plagiarism is the act of assisting or allowing another person to plagiarize or copy my/our work.

* Student(s) must complete all details except the faculty use part.

** Please submit all assignments to your course teacher or the office of the concerned teacher.

Group # 06

Sl No	Name	ID	PROGRAM	SIGNATURE
1	Mohammad Ansar Uddin	22-47975-2	BSc in CSE	
2	Chinmoy Guha	22-48056-2	BSc in CSE	
3	Nafiur Rahman Siam	23-50834-1	BSc in CSE	
4	Suvra Chakraborty	22-48067-2	BSc in CSE	
5	Arfanul Hoque Nehal	23-54270-3	BSc in CSE	
6	Mashkur Ibtesham Wazed	23-54266-3	BSc in CSE	

Faculty use only

FACULTY COMMENTS	Marks Obtained	
	Total Marks	

Experiment Title:

Familiarization with the Raspberry Pi

Objectives:

The objectives of this experiment are to-

- o Familiarize the students with the Raspberry Pi.

- o Implement an LED blink using the Raspberry Pi and its `time.sleep()` function.
- o Control the LEDs' ON/OFF using the input push switch.

- o Implement a traffic light control system.

Equipment List:

- 1) Activated Raspberry pi.
- 2) LED.
- 3) Push switch.
- 4) Resistor (220 ohm)
- 5) Breadboard
- 6) Jumper Wires.

Circuit Diagram:

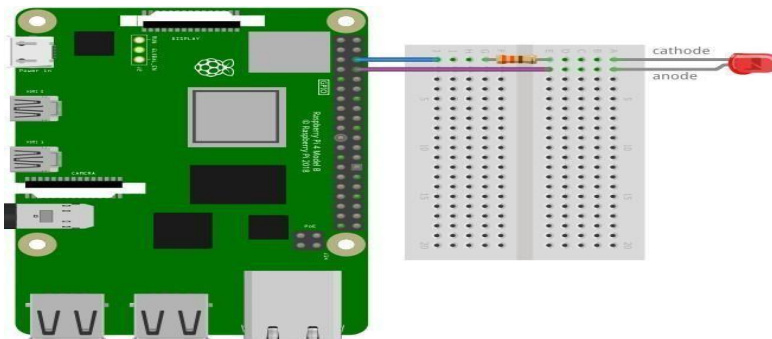


Fig. Experimental setup of the circuit for the LED blinking program using Raspberry Pi

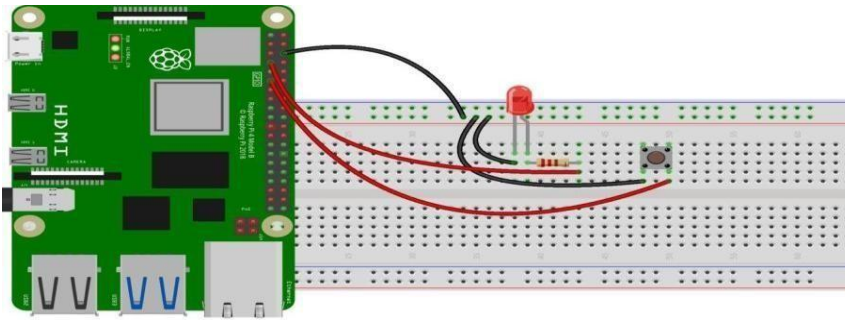


Fig. Experimental setup of the circuit for the LED controlling program with a button switch using Raspberry Pi

Code/Program:

The following is the code for the implementation of the LED blinking program with switch and without switch using Raspberry Pi with the necessary code explanation:

Lab Task # 1: LED Blinking

```
import time                                # time library contains the sleep()
GPIO.setmode(GPIO.BCM)                    # BCM pin numbering is used
GPIO.setwarnings(False)                   # to disable warnings
GPIO.setup(14, GPIO.OUT)                   # to set GPIO14 as an output
GPIO.output(14, GPIO.HIGH)                 # to specify the GPIO 14 as high print
"LED is ON"                               # show message to Terminal
time.sleep(2)                             # for two seconds
GPIO.output(14, GPIO.LOW)                  # to specify the GPIO 14 as low print "LED
is OFF"                                    # show message to Terminal
```

Lab Task # 2: LED controlling with a push button switch

The Python code for controlling the LED with the button on Raspberry Pi 4 is simple, we will first make a file with the name “LED_withButton.py” and open it with the nano editor:

```
from gpiozero import LED
#imports LED functions from gpiozero library from gpiozero import Button
#imports Button functions from gpiozero library

led = LED(4)
#declare the GPIO pin 4 for LED output and store it in led variable button =
Button(17)
#declare the GPIO pin 17 for Button output and store it in button variable while
True:
#initiated an infinite while loop()
    Button.wait_for_press()
#use the built-in function of the button to wait till press                led.on()
#turn on the led
    Button.wait_for_release()
#use the built-in function of button to wait till release                led.off()
#turn off the led
```

Lab Task # 3: Simple Traffic Control System

Design a traffic control system using RED, YELLOW, and GREEN LEDs.

```
#trafficlight import RPi.GPIO as GPIO import time
# setting-up the raspberrypi pins  GPIO.setmode(GPIO.BCM)
GPIO.setwarnings(False)
GPIO.setup(14,GPIO.OUT)
GPIO.setup(15,GPIO.OUT)
GPIO.setup(18,GPIO.OUT)

while (True):
    # for green led      GPIO.output(14,GPIO.HIGH)      print("Green LED is
ON")      time.sleep(3)      GPIO.output(14,GPIO.LOW)      time.sleep(2)
    print("Green LED is OFF")      # for yellow led      for i in range(3):
        GPIO.output(15,GPIO.HIGH)
        print("Yellow LED is ON for "+ str(i+1))
        time.sleep(0.5)      GPIO.output(15,GPIO.LOW)      time.sleep(0.5)
    print("Yellow LED is OFF for")

    # for red led      GPIO.output(18,GPIO.HIGH)      print ("Red LED is
ON")      time.sleep(5)      GPIO.output(18,GPIO.LOW)      print ("Red LED is
OFF")
    time.sleep(5)
```

Hardware Output Results:

Here is the hardware implementation of the LED blinking program with switch and without switch and the necessary explanation of the implementation:

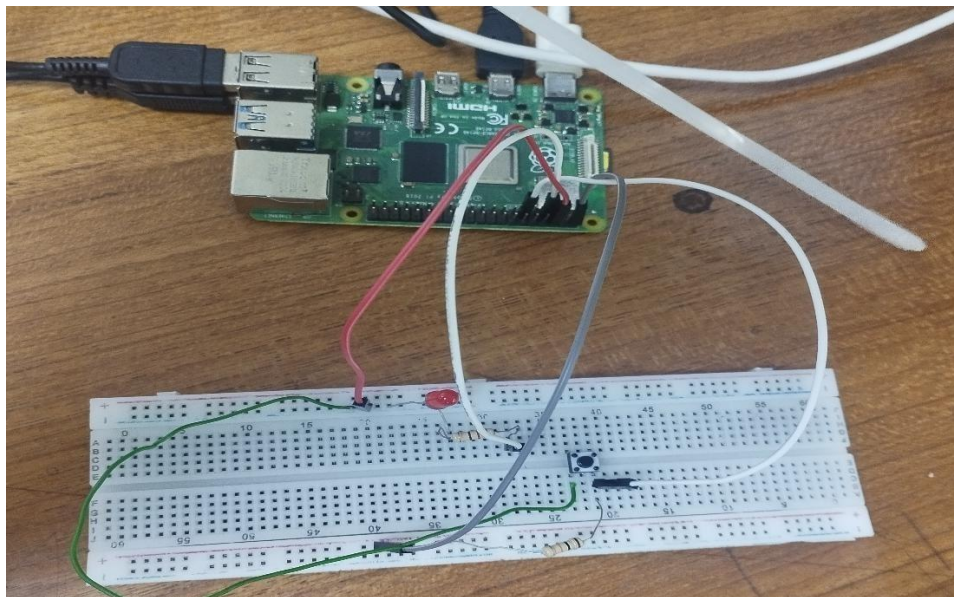


Fig. Hardware implementation of LED blinking with switch

Explanation:

In this experiment, a LED and a register were connected with the Raspberry Pi at GPIO4 pin and to ground the connection it connected to GND pin of the Raspberry Pi. For making a LED test with button, the button was connected with the GPIO17 pin and the LED was connected with GPIO4 pin. Both of them were connected with the GND pin of the Raspberry Pi.

Experimental Output Results:

Here is the hardware implementation of the LED blinking program without a switch and the necessary explanation of the implementation:

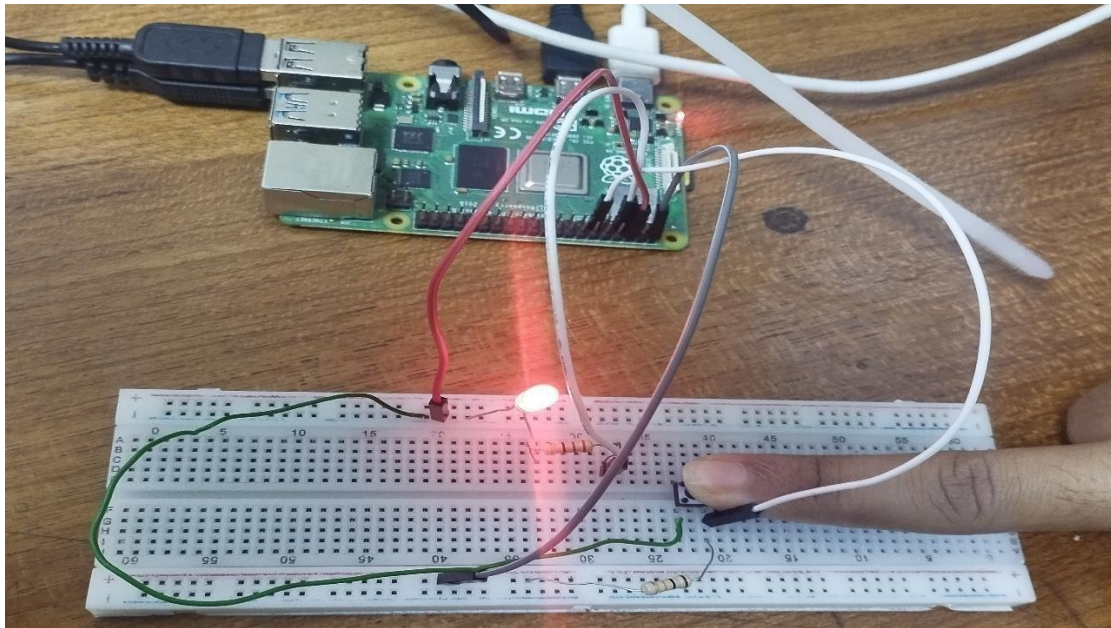
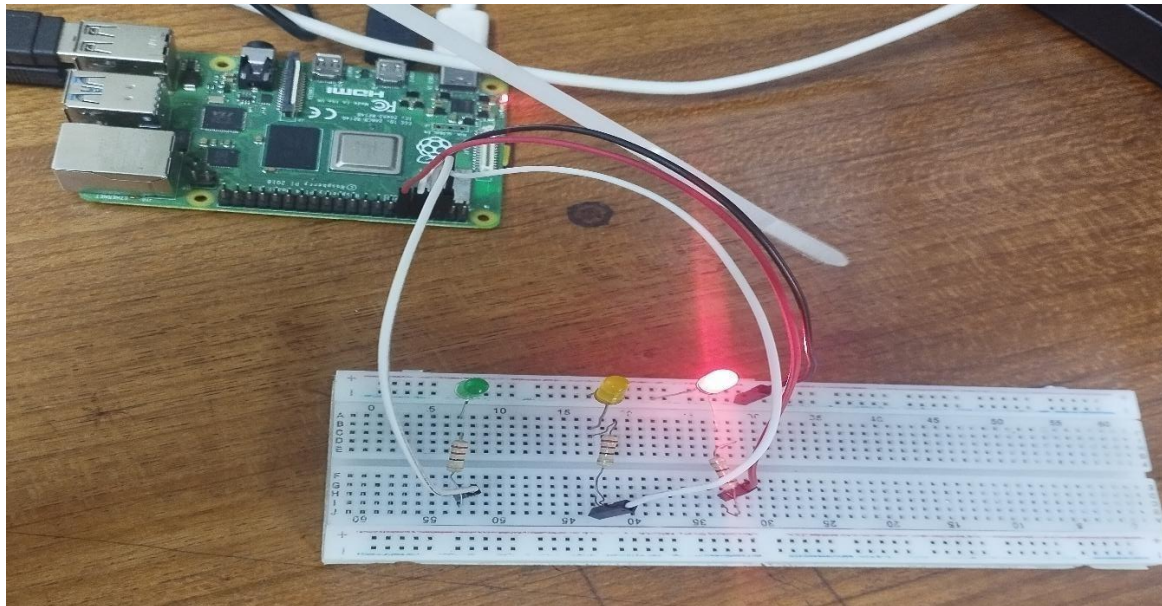
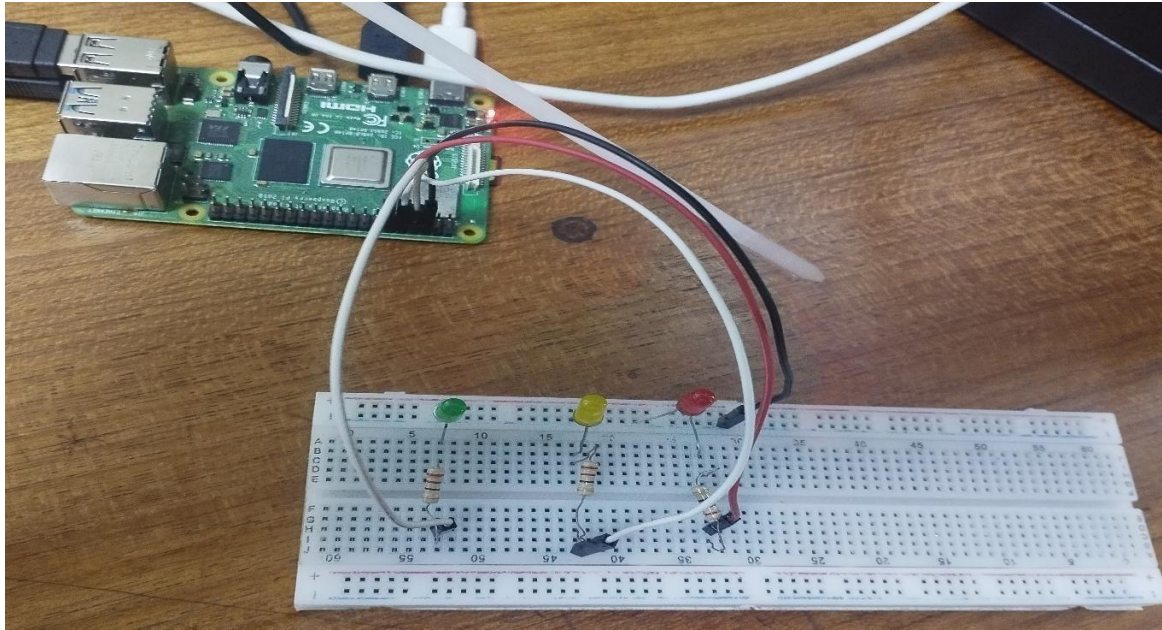


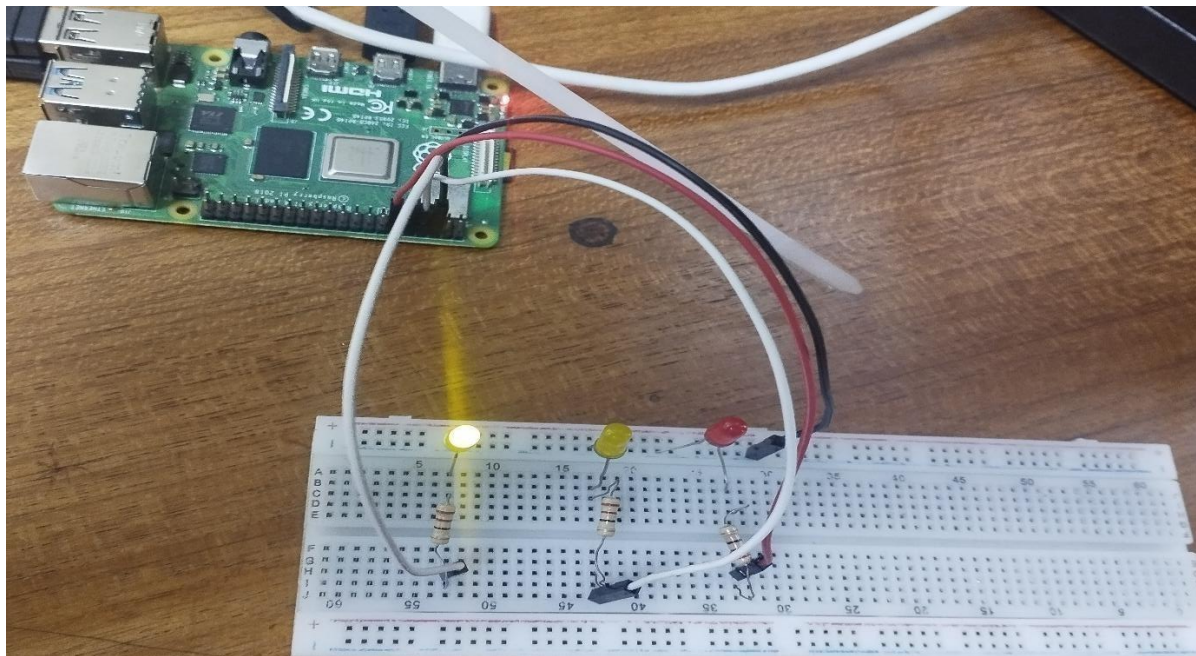
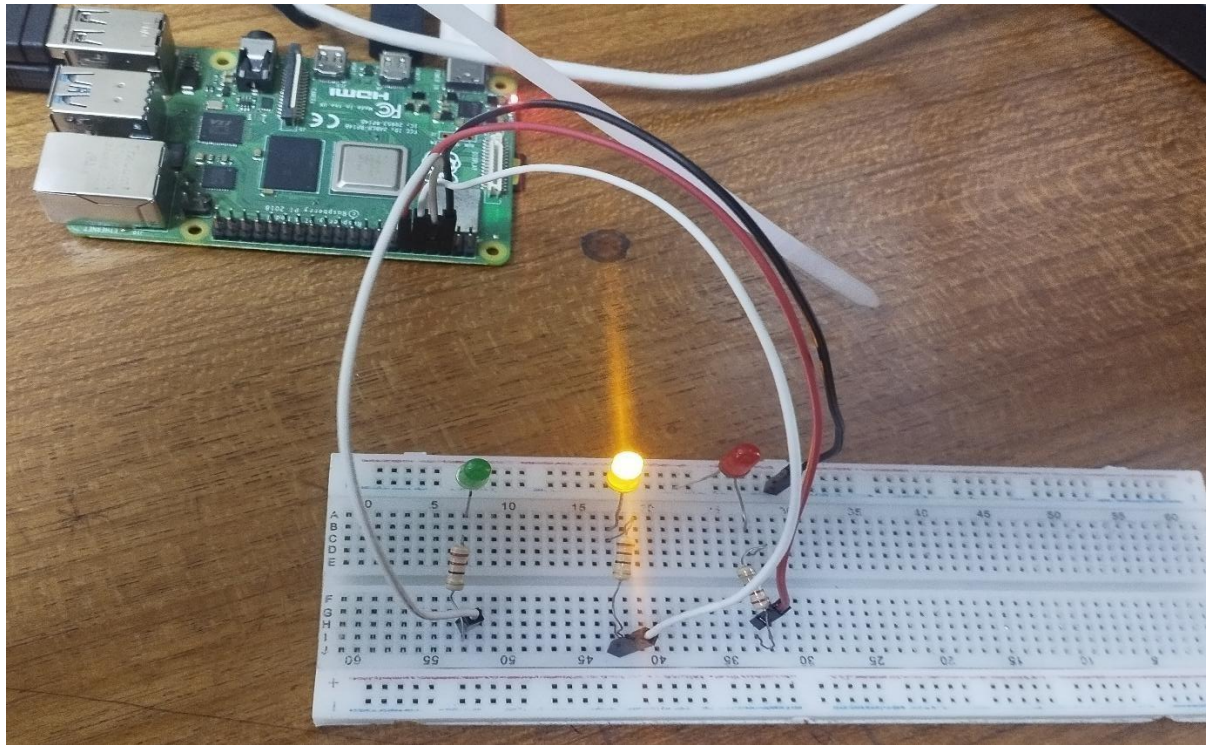
Fig. LED ON during Led blink program without switch on Raspberry Pi

Explanation:

In this experiment, when the circuit was built accordingly and code was ingested in the Raspberry Pi, the LED blinked after each 1 second. This meant that the GPIO4 pin was high for 1 second and low for 1 second. This duration was set using a delay function which can increase or decrease the delay of the time of blinking.

Here is the hardware implementation of the LED blinking program with switch and the necessary explanation of the implementation.





Explanation:

In this experiment, when the button was pressed, the GPIO 14 pin received a HIGH input which caused the program to let the Raspberry Pi to provide a HIGH output on the GPIO 4 pin that made the LED turn on. When the button was not pressed, the GPIO 14 pin was LOW and caused the GPIO 4 pin to provide a LOW output causing the LED to be OFF.

Simulation Output Results:

Here is the simulation implementation of the LED blinking program with switch, without switch and the simple traffic control system and the necessary explanation of the implementation:

Part 1: LED blink program without Switch:

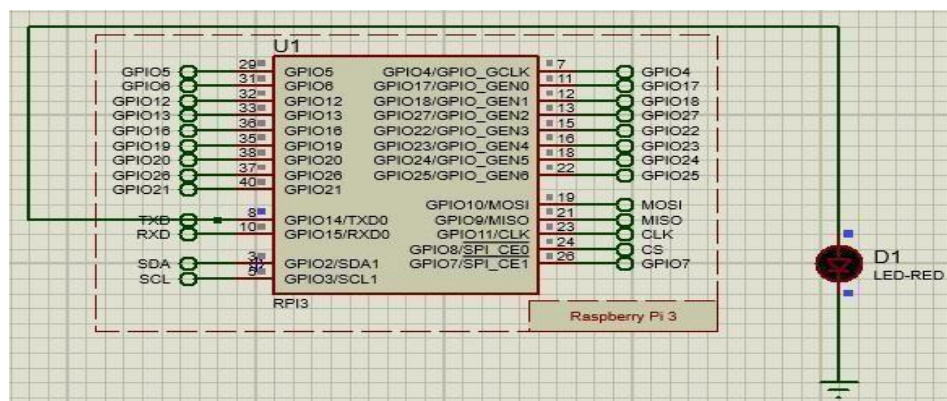


Fig. Led blink program without switch (LED is OFF)

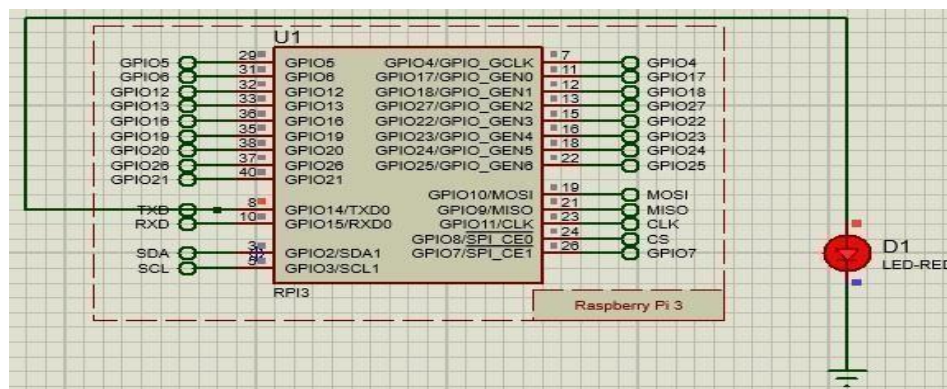


Fig. Led blink program without switch (LED is ON)

Part 2: LED blink program without Switch:

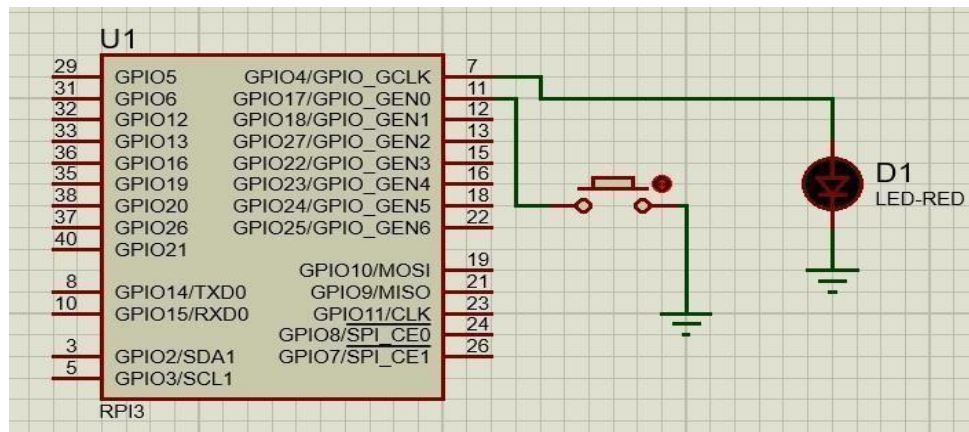


Fig. Led blink program with switch (LED is OFF)

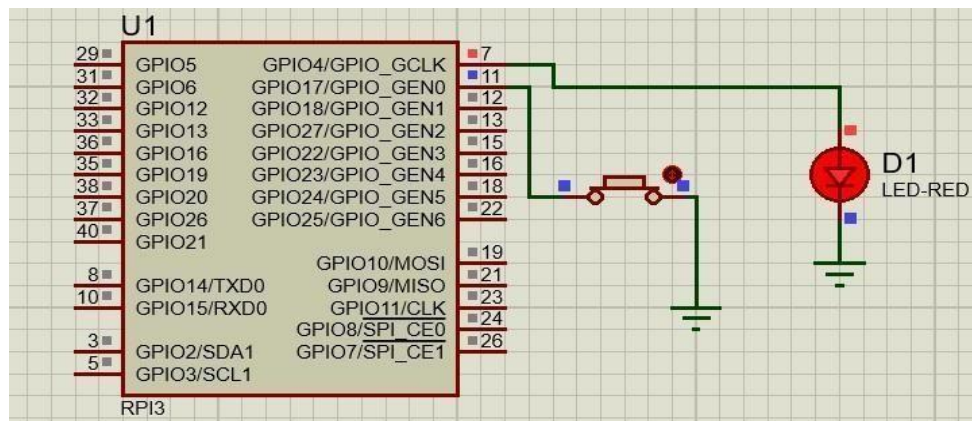


Fig. Led blink program with switch (LED is ON)

Part 3: Simple Traffic Control System:

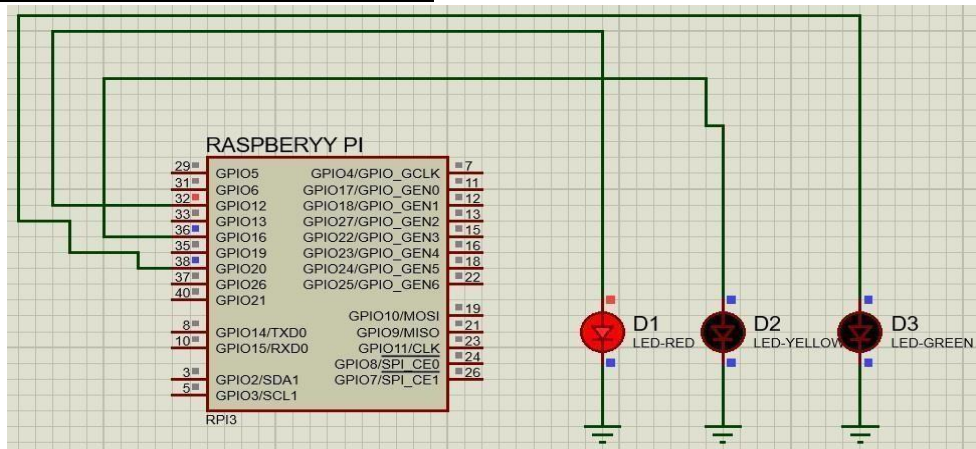


Fig. Simple Traffic Control System (Red LED is ON)

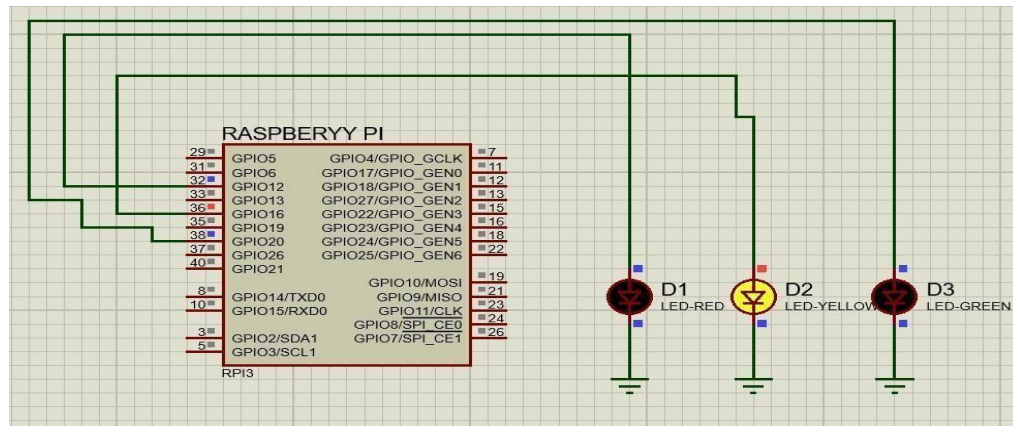


Fig. Simple Traffic Control System (Yellow LED is ON)

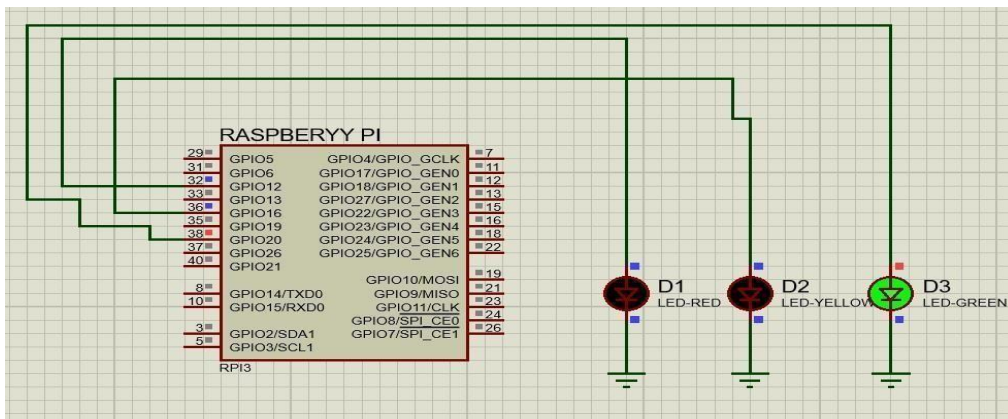


Fig. Simple Traffic Control System (Green LED is ON)

Explanation:

In this experiment, the circuits were developed on the Proteus Simulation software. All the components were loaded onto the developing canvas and the circuits were made accordingly. Afterward, the codes were written in Python. Then the Python scripts were loaded onto the Raspberry Pi. Then the simulations were conducted and the results were observed accordingly.

Discussion:

In this experiment, we learn how to use Raspberry Pi to create system which can control LED light and build a traffic management system. Through this experiment, we managed to have better understanding of Raspberry pi. We learn how we can import GPIO library and use it for our need in the simulator. We got familiar with Linux environment. The whole task was performed in an online simulator, which gives us the knowledge to run a similar Raspberry Pi task on online simulator. In future, this experiment will help us to build similar systems such as smart irrigation systems, gas leakage detector etc. using Raspberry pi. After that, the system was operated, and the systems operations were observed. The results were shown accordingly based on the formulas that were set on the code. All the results that were observed were carefully noted down for further evaluation. A similar system was developed on the simulation softwares like Proteus. The results that were obtained on the physical operation were evaluated with simulated outcomes. There were some Led Blinking discrepancies that were observed. The distance that was generated on the simulation's serial monitor were a bit different compared to the ones that were observed on the physical testing. This might be caused due to minor system and human errors. This caused the inconsistencies in the values of the serial monitor. Moreover, the detecting rate of the system in the physical environment and simulation virtual environment were a bit different. This was ruled as normal as human error was general in the physical world. From the observation it can be said that after both hardware and software implementation showed the expected outcomes, and the experimental objectives was achieved.

Reference(s):

[1] <https://www.raspberrypi.org/learn/>, accessed on 2nd July 2023.